

Walkthrough: Creating and Running Unit Tests for Managed Code

Visual Studio 2013 139 out of 180 rated this helpful

This walkthrough will step you through creating, running, and customizing a series of unit tests using the Microsoft unit test framework for managed code and the Visual Studio Test Explorer. You start with a C# project that is under development, create tests that exercise its code, run the tests, and examine the results. Then you can change your project code and re-run the tests.

This topic contains the following sections:

[Prepare the walkthrough](#)

[Create a unit test project](#)

[Create the test class](#)

- [Test class requirements](#)

[Create the first test method](#)

- [Test method requirements](#)

[Build and run the test](#)

[Fix your code and rerun your tests](#)

[Use unit tests to improve your code](#)

 Note
This walkthrough uses the Microsoft unit test framework for managed code. Test Explorer also can run tests from third party unit test frameworks that have adapters for Test Explorer. For more information, see How to: Install Third-Party Unit Test Frameworks
 Note
For information about how to run tests from a command line, see Walkthrough: Using the Command-line Test Utility .

Prerequisites

- The Bank project. See [Sample Project for Creating Unit Tests](#).

Prepare the walkthrough

To prepare the walkthrough

1. Open Visual Studio.
2. On the **File** menu, point to **New** and then click **Project**.

The **New Project** dialog box appears.
3. Under **Installed Templates**, click **Visual C#**.
4. In the list of application types, click **Class Library**.
5. In the **Name** box, type **Bank** and then click **OK**.

 Note
If the name "Bank" is already used, choose another name for the project.

The new Bank project is created and displayed in Solution Explorer with the Class1.cs file open in the Code Editor.

 Note
If the Class1.cs file is not open in the Code Editor, double-click the file Class1.cs in Solution Explorer to open it.

6. Copy the source code from the [Sample Project for Creating Unit Tests](#).
7. Replace the original contents of Class1.cs with the code from the [Sample Project for Creating Unit Tests](#).
8. Save the file as BankAccount.cs
9. On the **Build** menu, click **Build Solution**.

You now have a project named Bank. It contains source code to test and tools to test it with. The namespace for Bank, **BankAccountNS**, contains the public class **BankAccount**, whose methods you will test in the following procedures.

In this quick start, we focus on the **Debit** method. The Debit method is called when money is withdrawn an account and contains the following code:

C#

```
// method under test
public void Debit(double amount)
{
    if(amount > m_balance)
    {
        throw new ArgumentOutOfRangeException("amount");
    }
    if (amount < 0)
    {
        throw new ArgumentOutOfRangeException("amount");
    }
    m_balance += amount;
}
```

Create a unit test project

Prerequisite: Follow the steps in the procedure, [Prepare the Walkthrough](#).

To create a unit test project

1. On the **File** menu, choose **Add**, and then choose **New Project**
2. In the New Project dialog box, expand **Installed**, expand **Visual C#**, and then choose **Test**.
3. From the list of templates, select **Unit Test Project**.
4. In the **Name** box, enter BankTest, and then choose **OK**.

The **BankTests** project is added to the the **Bank** solution.

5. In the **BankTests** project, add a reference to the **Bank** solution.

In Solution Explorer, select **References** in the **BankTests** project and then choose **Add Reference...** from the context menu.

6. In the Reference Manager dialog box, expand **Solution** and then check the **Bank** item.

Create the test class

We need a test class for verifying the **BankAccount** class. We can use the UnitTest1.cs that was generated by the project template, but we should give the file and class more descriptive names. We can do that in one step by renaming the file in Solution Explorer.

Renaming a class file

In Solution Explorer, select the UnitTest1.cs file in the BankTests project. From the context menu, choose **Rename**, and then rename the file to BankAccountTests.cs. Choose **Yes** on the dialog that asks if you want to rename all references in the project to the code element 'UnitTest1'. This step changes the name of the class to **BankAccountTest**.

The BankAccountTests.cs file now contains the following code:

C#

```
// unit test code
using System;
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace BankTests
{
    [TestClass]
    public class BankAccountTests
    {
        [TestMethod]
        public void TestMethod1()
        {
        }
    }
}
```

Add a using statement to the project under test

We can also add a using statement to the class to let us to call into the project under test without using fully qualified names. At the top of the class file, add:

C#

```
using BankAccountNS
```

Test class requirements

The minimum requirements for a test class are the following:

- The `[TestClass]` attribute is required in the Microsoft unit testing framework for managed code for any class that contains unit test methods that you want to run in Test Explorer.
- Each test method that you want Test Explorer to run must have the `[TestMethod]` attribute.

You can have other classes in a unit test project that do not have the `[TestClass]` attribute, and you can have other methods in test classes that do not have the `[TestMethod]` attribute. You can use these other classes and methods in your test methods.

Create the first test method

In this procedure, we will write unit test methods to verify the behavior of the `Debit` method of the `BankAccount` class. The method is listed above.

By analyzing the method under test, we determine that there are at least three behaviors that need to be checked:

1. The method throws an `ArgumentOutOfRangeException` if the credit amount is greater than the balance.
2. It also throws `ArgumentOutOfRangeException` if the credit amount is less than zero.
3. If the checks in 1.) and 2.) are satisfied, the method subtracts the amount from the account balance.

In our first test, we verify that that a valid amount (one that is less than the account balance and that is greater than zero) withdraws the correct amount from the account.

To create a test method

1. Add a using `BankAccountNS;` statement to the `BankAccountTests.cs` file.
2. Add the following method to that `BankAccountTests` class:

```
C#
// unit test code
[TestMethod]
public void Debit_WithValidAmount_UpdatesBalance()
{
    // arrange
    double beginningBalance = 11.99;
    double debitAmount = 4.55;
    double expected = 7.44;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // act
    account.Debit(debitAmount);

    // assert
    double actual = account.Balance;
    Assert.AreEqual(expected, actual, 0.001, "Account not debited correctly");
}
```

The method is rather simple. We set up a new `BankAccount` object with a beginning balance and then withdraw a valid amount. We use the Microsoft unit test framework for managed code `AreEqual` method to verify that the ending balance is what we expect.

Test method requirements

A test method must meet the following requirements:

- The method must be decorated with the `[TestMethod]` attribute.
- The method must return `void`.
- The method cannot have parameters.

Build and run the test

To build and run the test

1. On the **Build** menu, choose **Build Solution**.

If there are no errors, the `UnitTestExplorer` window appears with `Debit_WithValidAmount_UpdatesBalance` listed in the **Not Run Tests** group. If Test Explorer does not appear after a successful build, choose **Test** on the menu, then choose **Windows**, and then choose **Test Explorer**.

2. Choose **Run All** to run the test. As the test is running the status bar at the top of the window is animated. At the end of the test run, the bar turns green if all the test methods pass, or red if any of the tests fail.
3. In this case, the test does fail. The test method is moved to the **Failed Tests** group. Select the method in Test Explorer to view the details at the bottom of the window.

Fix your code and rerun your tests

Analyze the test results

The test result contains a message that describes the failure. For the `AreEquals` method, message displays you what was expected (the (**Expected<XXX>** parameter) and what was actually received (the **Actual<YYY>** parameter). We were expecting the balance to decline from the beginning balance, but instead it has increased by the amount of the withdrawal.

A reexamination of the Debit code shows that the unit test has succeeded in finding a bug. The amount of the withdrawal is added to the account balance when it should be subtracted.

Correct the bug

To correct the error, simply replace the line

C#

```
m_balance += amount;
```

with

C#

```
m_balance -= amount;
```

Rerun the test

In Test Explorer, choose **Run All** to rerun the test. The red/green bar turns green, and the test is moved to the **Passed Tests** group.

Use unit tests to improve your code

This section describes how an iterative process of analysis, unit test development, and refactoring can help you make your production code more robust and effective.

Analyze the issues

After creating a test method to confirm that a valid amount is correctly deducted in the `Debit` method, we can turn to remaining cases in our original analysis:

1. The method throws an `ArgumentOutOfRangeException` if the credit amount is greater than the balance.
2. It also throws `ArgumentOutOfRangeException` if the credit amount is less than zero.

Create the test methods

A first attempt at creating a test method to address these issues seems promising:

C#

```
//unit test method
[TestMethod]
[ExpectedException(typeof(ArgumentOutOfRangeException))]
public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRangeException()
{
    // arrange
    double beginningBalance = 11.99;
    double debitAmount = -100.00;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // act
    account.Debit(debitAmount);

    // assert is handled by ExpectedException
}
```

We use the `ExpectedExceptionAttribute` attribute to assert that the right exception has been thrown. The attribute causes the test to fail unless an `ArgumentOutOfRangeException` is thrown. Running the test with both positive and negative `debitAmount` values and then temporarily modifying the method under test to throw a generic `ApplicationException` when the amount is less than zero demonstrates that test behaves correctly. To test the case when the amount withdrawn is greater than the balance, all we need to do is:

1. Create a new test method named `Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException`.
2. Copy the method body from `Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRangeException` to the new method.
3. Set the `debitAmount` to a number greater than the balance.

Run the tests

Running the two methods with different values for `debitAmount` demonstrates that the tests adequately handle our remaining cases. Running all three tests confirm that all cases in our original analysis are correctly covered.

Continue the analysis

However, the last two test methods are also somewhat troubling. We cannot be certain which condition in the code under test throws when either test runs. Some way of differentiating the two conditions would be helpful. As we think about the problem more, it becomes apparent that knowing which condition was violated would increase our confidence in the tests.

This information would also very likely be helpful to the production mechanism that handles the exception when it is thrown by the method under test. Generating more information when the method throws would assist all concerned, but the `ExpectedException` attribute cannot supply this information..

Looking at the method under test again, we see both conditional statements use an `ArgumentOutOfRangeException` constructor that takes name of the argument as a parameter:

C#

```
throw new ArgumentOutOfRangeException("amount");
```

From a search of the MSDN Library, we discover that a constructor exists that reports far richer information. `ArgumentOutOfRangeException(String, Object, String)` includes the name of the argument, the argument value, and a user-defined message. We can refactor the method under test to use this constructor. Even better, we can use publicly available type members to specify the errors.

Refactor the code under test

We first define two constants for the error messages at class scope:

C#

```
// class under test
public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds balance";
public const string DebitAmountLessThanZeroMessage = "Debit amount less than zero";
```

We then modify the two conditional statements in the `Debit` method:

C#

```
// method under test
// ...
if (amount > m_balance)
{
    throw new ArgumentOutOfRangeException("amount", amount, DebitAmountExceedsBalanceMessage);
}

if (amount < 0)
{
    throw new ArgumentOutOfRangeException("amount", amount, DebitAmountLessThanZeroMessage);
}
// ...
```

Refactor the test methods

In our test method, we first remove the `ExpectedException` attribute. In its place, we catch the thrown exception and verify that it was thrown in the correct condition statement. However, we must now decide between two options to verify our remaining conditions. For example in the `Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException` method, we can take one of the following actions:

- Assert that the `ActualValue` property of the exception (the second parameter of the `ArgumentOutOfRangeException` constructor) is greater than the beginning balance. This option requires that we test the `ActualValue` property of the exception against the `beginningBalance` variable of the test method, and also requires then verify that the `ActualValue` is greater than zero.
- Assert that the message (the third parameter of the constructor) includes the `DebitAmountExceedsBalanceMessage` defined in the `BankAccount` class.

The `StringAssert.Contains` method in the Microsoft unit test framework enables us to verify the second option without the calculations that are required of the first option.

A second attempt at revising `Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException` might look like:

C#

```
[TestMethod]
public void Debit_WhenAmountIsGreaterThanBalance_ShouldThrowArgumentOutOfRangeException()
{
    // arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.0;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // act
    try
    {
        account.Debit(debitAmount);
    }
    catch (ArgumentOutOfRangeException e)
    {
        // assert
        StringAssert.Contains(e.Message, BankAccount.DebitAmountExceedsBalanceMessage);
    }
}
```

Retest, rewrite, and reanalyze

When we retest the test methods with different values, we encounter the following facts:

1. If we catch the correct error by using an `debitAmount` that is greater than the balance, the `Contains` assert passes, the exception is ignored, and so the test method passes. This is the behavior we want.

2. If we use an `debitAmount`, the assert fails because the wrong error message is returned. The assert also fails if we introduce a temporary `ArgumentOutOfRangeException` exception at another point in the method under test code path. This too is good.
3. If the `debitAmount` value is valid (i.e., less than the balance but greater than zero, no exception is caught, so the assert is never caught. The test method passes. This is not good, because we want the test method to fail if no exception is thrown.

The third fact is a bug in our test method. To attempt to resolve the issue, we add a `Fail` assert at the end of the test method to handle the case where no exception is thrown.

But retesting shows that the test now fails if the correct exception is caught. The catch statement resets the exception and the method continues to execute, failing at the new assert. To resolve the new problem, we add a `return` statement after the `StringAssert`. Retesting confirms that we have fixed our problems. Our final version of the `Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException` looks like the following:

```
C#  
  
[TestMethod]  
public void Debit_WhenAmountIsGreaterThanBalance_ShouldThrowArgumentOutOfRangeException()  
{  
    // arrange  
    double beginningBalance = 11.99;  
    double debitAmount = 20.0;  
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);\n\n    // act  
    try  
    {  
        account.Debit(debitAmount);\n    }  
    catch (ArgumentOutOfRangeException e)  
    {  
        // assert  
        StringAssert.Contains(e.Message, BankAccount.DebitAmountExceedsBalanceMessage);\n        return;\n    }\n    Assert.Fail("No exception was thrown.")\n}
```

In this final section, the work that we did improving our test code led to more robust and informative test methods. But more importantly, the extra analysis also led to better code in our project under test.